

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018



IDT Project Report

on

“SMART COLOR DETECTION FOR COLOR BLIND”

- Submitted in partial fulfillment of the requirements for the **second Semester of the Bachelor of Engineering Degree**, towards the completion of the **IDT Project** under the **Interdisciplinary Project Laboratory**, Department of Basic Sciences.

By

MR. MANJUNATHA BABU B

SN	Name	USN/Roll number
1.	BED PRASAD ARYAL	1CR25CS026
2.	BAVYESH MATTOO	1CR25CS025
3.	ARPITA DUNDI	1CR25EC025
4.	ARSH KHAN	1CR25EC026

Under the Guidance of
Mr. MANJUNATH BABU B
Assoc. Professor



CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

- #132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,
- BANGALORE-560037

DEPARTMENT OF BASIC SCIENCES



CERTIFICATE

This is to certify that the File Structures project entitled “**SMART COLOR DETECTOR FOR COLOR BLIND**” Has been successfully carried out by BED PRASAD, BAVYESH MATTOO, ARPITA DUNDI, ARSH KHAN Bonafide students of **CMR Institute of Technology**.

The project is submitted in partial fulfillment of the requirements for the First Semester of the Bachelor of Engineering Degree, towards the completion of the IDT Project under the **Interdisciplinary Project Laboratory, Department of Basic Sciences**. It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This File Structures mini project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

Signature of Guide

Mr. MANJUNATHA BABU B

Assoc. professor

Department of AIDS

CMRIT

Signature of HoD

Dr. Raveesha

HoD, Dept of Physics

CMRIT

External Viva

Name of the examiners

Signature with date

1.

2.

ACKNOWLEDGEMENT

I sincerely express my gratitude to Dr. Sanjay Jain, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to Dr. Raveesha K H, HoD, Dept. of Physics, CMRIT for his valuable guidance and support.

I am especially grateful to my internal guide, Mr. MANJUNATHA BABU B, Associate Professor, for his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

BED PRASAD ARYAL- 1CR25CS026

BAVYESH MATTOO -1CR25CS025

ARPITA DUNDI- 1CR25EC025

ARSH KHAN -1CR25EC026

Abstract

The Smart Color Detector for Colour Blind People is a software-based assistive system designed to help visually impaired and color blind individuals identify colors in real time. The system uses a webcam to capture live video frames and processes the image using the OpenCV library in Python. A specific area from the video frame is analysed to extract RGB color values, and based on predefined conditions, the software identifies the detected color.

After detecting the color, the system displays the color name and RGB values on the screen and announces the detected color through voice output using the pyttsx3 text-to-speech library. This makes the application more accessible and user-friendly for people who face difficulty distinguishing colors in daily life.

The project aims to provide a simple, low-cost, and effective solution that can improve independence and confidence for color blind users.

The system is easy to use, works in real time, and can be further enhanced with advanced color recognition and mobile integration in the future.

Keywords:

Color Detection, Color Blind Assistance, OpenCV, Python, RGB Analysis, Text-to-Speech, pyttsx3, Webcam Processing, Computer Vision, Assistive Technology, Real-Time Detection, Smart Color Detector.

CHAPTER 1

Introduction

The Smart Color Detector is a software application developed to help color blind people identify colors easily and accurately. Many people with color blindness face difficulty distinguishing between certain colors in daily life, which can create confusion and inconvenience.

This project uses a webcam to capture live video and processes the image using OpenCV in Python. The system detects the color from a selected area by analyzing RGB values. After identifying the color, the software displays the color name on the screen and also announces it through voice output using the pyttsx3 text-to-speech library.

The main aim of this project is to provide a simple, low-cost, and user-friendly assistive tool that improves accessibility and helps color blind individuals become more independent in their everyday activities.

Modern Developments

Modern developments in computer vision and artificial intelligence have made color detection systems more accurate and accessible. Technologies like Python, OpenCV, and text-to-speech systems are now widely used to create smart assistive applications for visually impaired and color blind people.

Today, real-time color detection software can identify colors quickly using webcams and image processing techniques. Voice assistance features also help users hear the detected color, making the system more user-friendly and effective. The Smart Color Detector project is an example of these modern technologies being used to improve accessibility and daily life for color blind individuals.

CHAPTER 2

Problem Statement

2.1 Description

Color blindness represents a significant visual impairment where individuals struggle to distinguish between specific hues. Such limitations manifest as daily obstacles, complicating tasks like object identification, wardrobe selection, navigating traffic signals, or interpreting color-coded data. Current technological aids are often prohibitively expensive or lack practical portability for consistent application.

The Smart Color Detector initiative addresses these concerns by leveraging webcam hardware and advanced image processing to facilitate real-time color recognition. By calculating RGB coordinates, the system delivers immediate text-based and auditory feedback, empowering those with color vision deficiencies to navigate their environments with greater autonomy.

2.2 Challenge Statement

A primary technical hurdle involves maintaining detection precision across diverse environmental lighting while ensuring low-latency processing. The architecture must robustly handle live video streams, performing RGB analysis and categorization with high fidelity and minimal margin for error.

Furthermore, the software must balance sophisticated functionality with a highly accessible, intuitive interface. Achieving seamless voice synthesis and efficient computational performance on standard hardware remains critical to delivering a viable assistive experience for the target demographic.

2.3 Proposed Solution

The suggested framework involves the engineering of a Smart Color Detector application intended to assist those with color vision deficiencies via live chromatic recognition. This architecture utilizes webcam hardware for video stream acquisition, subsequently employing the OpenCV library within a Python environment for digital image manipulation.

By isolating a specific region of interest for RGB coordinate computation, the algorithm successfully categorizes the intended hue. Upon identification, the interface presents the nomenclature and color data visually while simultaneously providing auditory confirmation through the pyttsx3 synthesis engine, ensuring comprehensive accessibility.

This conceptualized system offers a low-latency, economical, and intuitive methodology for precise visual assistance. By delivering rapid chromatic feedback, the project facilitates improved autonomy for users navigating the complexities of color-dependent environments in their daily lives.

CHAPTER 3

DESIGN

3.1 Design Thinking Process

Empathize

During this initial phase, the specific obstacles encountered by individuals with color vision deficiencies were rigorously analyzed. Such users frequently experience significant friction when performing routine tasks, including garment coordination, interpreting traffic signals, analyzing color-coded diagrams, or identifying botanical and medicinal items. These persistent difficulties often lead to a reduction in personal confidence and an increased reliance on external assistance. Comprehensive research into these lived experiences underscored the critical requirement for a streamlined and efficient technological intervention.

Define

Following the investigation into user constraints, the core objective was established with technical precision. The project is dedicated to engineering a software framework capable of high-fidelity, real-time chromatic recognition to empower color blind users. The architecture is designed to be economical, intuitive, and highly responsive, ensuring broad accessibility. By integrating simultaneous visual telemetry and auditory synthesis, the system enables users to navigate color-dependent environments with enhanced autonomy and reduced cognitive load.

Ideate

In this conceptual stage, various methodologies and technological stacks were evaluated to determine the optimal deployment strategy. Extensive research into digital image processing led to the selection of the Python environment, favored for its computational efficiency and robust library support. The framework incorporates OpenCV for webcam stream management and image manipulation, alongside the

pyttsx3 engine for voice synthesis. This synthesized approach facilitates the acquisition of live video for RGB coordinate analysis and immediate audio confirmation.

Prototype

A functional iteration of the Smart Color Detector was developed to demonstrate real-time viability. The application utilizes webcam hardware to continuously capture video frames, isolating a specific region of interest for RGB value computation. Utilizing a logic-based categorization system, the software determines the corresponding hue and presents the nomenclature on the interface. Concurrently, the system triggers a text-to-speech module for auditory feedback, maintaining a lightweight and user-centric architecture for seamless operation.

Test

The integrated system underwent rigorous validation using a diverse array of chromatic samples across varying luminance levels to ensure detection accuracy. This evaluative process was instrumental in refining the classification algorithms and minimizing latency. Critical parameters, including voice synthesis clarity and computational performance, were scrutinized to optimize the end-user experience. Following iterative enhancements, the system demonstrated reliable performance, establishing itself as a viable assistive tool for the target demographic.

3.2 Methodology

The methodology of the Smart Color Detector project involves capturing live video, processing image data, detecting colors, and providing output to the user through both text and voice assistance.

Step 1: Video Capture

The system uses a webcam to capture live video frames continuously. OpenCV is used to access the camera and process the incoming video in real time.

Step 2: Frame Processing

Each video frame is processed by selecting a small detection area at the center of the screen. This area is used for analyzing the color of the object placed in front of the camera.

Step 3: RGB Value Extraction

The system calculates the average RGB (Red, Green, Blue) values from the selected detection area. These RGB values represent the color information of the object.

Step 4: Color Detection

The extracted RGB values are compared with predefined color ranges stored in the program. Based on this comparison, the software identifies the closest matching color.

Step 5: Display Output

After detecting the color, the system displays the color name and RGB values on the screen so the user can visually view the result.

Step 6: Voice Output

Using the pyttsx3 text-to-speech library, the detected color name is announced through the speaker. This helps color blind users identify colors more easily.

Step 7: System Exit

The program continues detecting colors in real time until the user presses a specific key (such as 'Q') to close the application safely.

3.3.1 Materials Used

The development of the Smart Color Detector required a combination of software tools and hardware components. These materials were selected to ensure real-time performance, simplicity, and ease of use for color blind individuals.

1. Python Programming Language –

Python is the main programming language used to develop the Smart Color Detector system. It provides simple syntax and powerful libraries for image processing and voice assistance features.

2. OpenCV Library –

OpenCV is used for capturing live video through the webcam and processing image frames. It helps in detecting colors by analyzing RGB values from the selected area of the image.

3. pyttsx3 Library –

pyttsx3 is a text-to-speech library used to provide voice output. It announces the detected color through the speaker, making the system more accessible for users.

4. NumPy Library –

NumPy is used for numerical operations and handling RGB value calculations efficiently during image processing.

5. Webcam –

A webcam is used to capture real-time video input for detecting colors. The camera continuously sends image frames to the software for processing.

6. Computer System (Hardware) –

A standard computer or laptop is used to run the application and process the image data.

Specifications:

Processor: Intel / AMD (minimum dual-core)

RAM: 4 GB or higher

Operating System: Windows / Linux

Storage: Minimum 5 GB free space

7. Code Editor / IDE –

An IDE such as Visual Studio Code, PyCharm, or IDLE is used to write, test, and debug the Python program efficiently.

Chapter 4

```
import cv2

import pytsx3

# Initialize voice engine

engine = pytsx3.init()

# Store previous detected color

last_color = ""

# Function to detect color

def detect_color(r, g, b):

    if r > 150 and g < 100 and b < 100:

        return "RED", (0, 0, 255)

    elif g > 150 and r < 100 and b < 100:

        return "GREEN", (0, 255, 0)

    elif b > 150 and r < 100 and g < 100:

        return "BLUE", (255, 0, 0)

    elif r > 150 and g > 150 and b < 100:
```

```
    return "YELLOW", (0, 255, 255)
```

```
elif r > 150 and g > 150 and b > 150:
```

```
    return "WHITE", (255, 255, 255)
```

```
elif r < 50 and g < 50 and b < 50:
```

```
    return "BLACK", (80, 80, 80)
```

```
else:
```

```
    return "UNKNOWN", (255, 0, 255)
```

```
# Open webcam
```

```
cap = cv2.VideoCapture(0)
```

```
# Check camera
```

```
if not cap.isOpened():
```

```
    print("Cannot open camera")
```

```
    exit()
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
if not ret:  
  
    print("Failed to access camera")  
  
    break
```

```
# Get frame dimensions  
  
height, width, _ = frame.shape
```

```
# Bigger center box  
  
box_size = 150
```

```
x1 = width // 2 - box_size  
  
y1 = height // 2 - box_size
```

```
x2 = width // 2 + box_size  
  
y2 = height // 2 + box_size
```

```
# Draw detection box  
  
cv2.rectangle(frame,  
               (x1, y1),  
               (x2, y2),  
               (0, 255, 0),  
               3)
```

```
# Region inside box
```

```
roi = frame[y1:y2, x1:x2]

# Calculate average BGR values
avg_color = roi.mean(axis=0).mean(axis=0)

blue = int(avg_color[0])
green = int(avg_color[1])
red = int(avg_color[2])

# Detect color
color_name, text_color = detect_color(red, green, blue)

# Speak only when color changes
if color_name != last_color and color_name != "UNKNOWN":

    print("Detected:", color_name)

    engine.stop()

    engine.say(f"Detected color is {color_name}")
    engine.runAndWait()

    last_color = color_name
```

```
# RGB text

rgb_text = f'R:{red} G:{green} B:{blue}'

cv2.putText(frame,

             rgb_text,

             (40, 110),

             cv2.FONT_HERSHEY_SIMPLEX,

             0.8,

             (255, 255, 255),

             2)

# Display detected color name

cv2.putText(frame,

            f'COLOR: {color_name}',

            (40, 60),

            cv2.FONT_HERSHEY_SIMPLEX,

            1.2,

            text_color,

            3)

# Project title

cv2.putText(frame,

            "SMART COLOR DETECTOR",

            (40, 420),
```

```
        cv2.FONT_HERSHEY_SIMPLEX,

        0.9,

        (0, 255, 255),

        2)

# Exit instruction

cv2.putText(frame,

            "Press Q to Exit",

            (40, 460),

            cv2.FONT_HERSHEY_SIMPLEX,

            0.7,

            (200, 200, 200),

            2)

# Show webcam window

cv2.imshow("Smart Color Detector", frame)

# Quit on Q

if cv2.waitKey(1) & 0xFF == ord('q'):

    break

# Release everything

cap.release()

cv2.destroyAllWindows()
```

Chapter 5

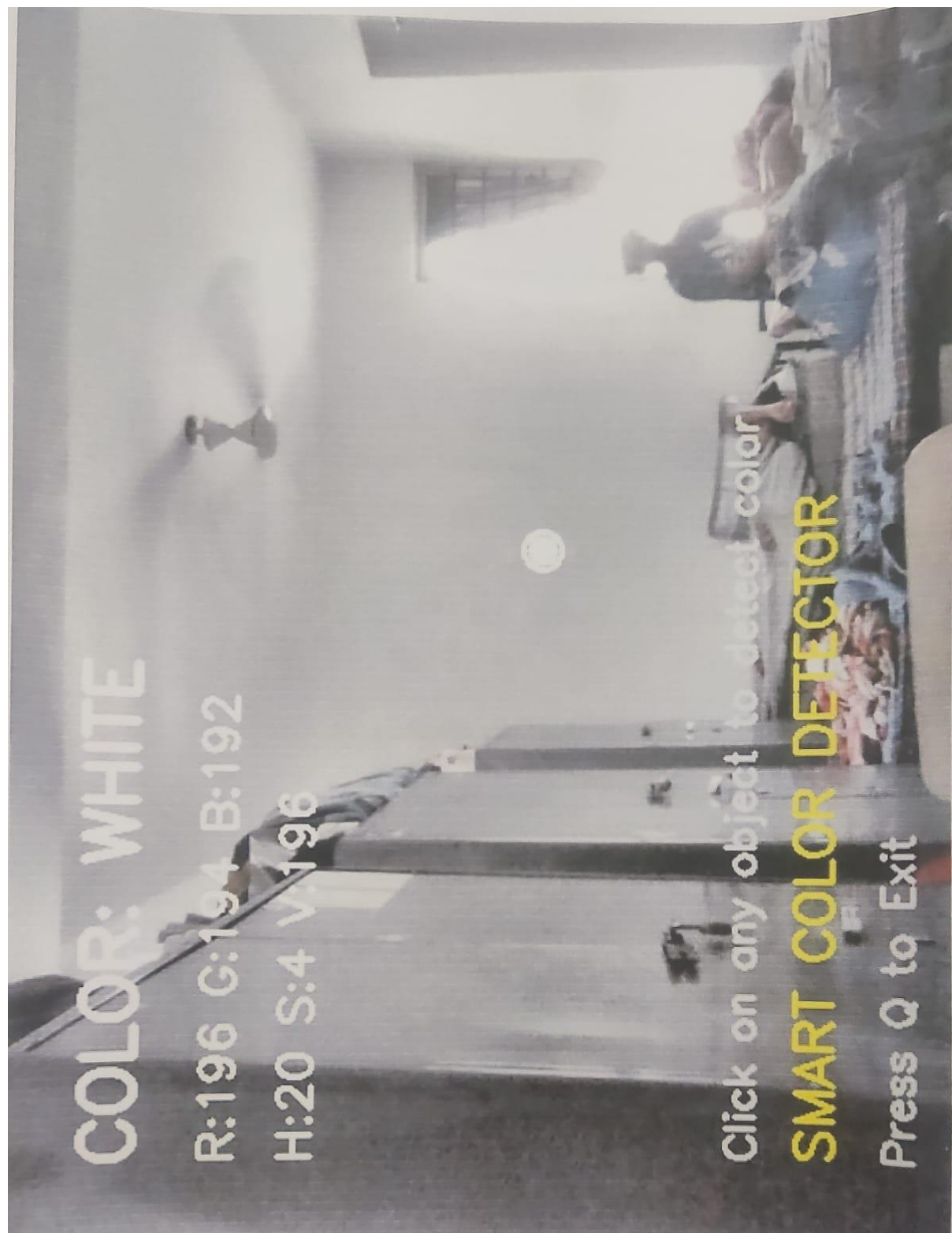
Results and Analysis

User Testing & Feedback

The Smart Color Detector system was tested using different colored objects under various lighting conditions to evaluate its accuracy, performance, and ease of use. User testing was conducted by showing objects of different colors in front of the webcam and checking whether the system correctly identified and announced the detected color.

The software successfully detected major colors such as red, green, blue, yellow, black, and white with good accuracy. The voice output feature worked properly and helped users identify colors easily through audio feedback. The system also displayed the detected color name and RGB values clearly on the screen.

Feedback from testing showed that the system is simple, user-friendly, and useful for assisting color blind individuals in everyday activities. Some improvements were suggested for detecting a wider range of colors and improving performance under low-light conditions.



CHAPTER 6

Conclusion & Future Scope

Conclusion

The Smart Color Detector initiative has achieved its objective of engineering a real-time assistive framework for individuals with color vision deficiencies. By integrating webcam hardware with OpenCV digital image manipulation and auditory synthesis, the system successfully categorizes and announces chromatic data with high fidelity. This architecture provides an economical and intuitive methodology that enhances user autonomy and accessibility during routine daily operations.

Future Work

Subsequent iterations of this system may incorporate sophisticated algorithms to identify a broader spectrum of nuanced hues and advanced chromatic recognition models. Transitioning the framework into a mobile-centric application would significantly increase portability and practical application. Furthermore, the integration of artificial intelligence for object categorization, multilingual voice synthesis, and improved low-light performance will further optimize the system's computational efficacy and target demographic utility.

Bibliography

Python Software Foundation. Documentation for the Python Programming Language.

OpenCV Documentation. Open Source Computer Vision Technical Library.

Pytsx3 Documentation. Text-to-Speech Engine and Conversion Library.

NumPy Documentation. Numerical Computing Framework for Python Environments.

Rosebrock, Adrian. Digital Image Manipulation and Computer Vision with OpenCV.

Peer-reviewed research and online scholarly articles regarding real-time chromatic recognition.

